



SAPIENZA
UNIVERSITÀ DI ROMA

Evaluation of Statistical Methods for Generative Password-guessing models

Facoltà di Ingegneria dell'informazione, informatica e statistica
Informatica

Roberto Di Rosa

ID number 2043388

Advisor

Prof. De Gaspari

Co-Advisor

Academic Year 2025/2026

Evaluation of Statistical Methods for Generative Password-guessing models
Sapienza University of Rome

© 2025 Roberto Di Rosa. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: dirosa.2043388@studenti.uniroma1.it

Abstract

"Don't be overwise; fling yourself straight into life, without deliberation; don't be afraid - the flood will bear you to the bank and set you safe on your feet again."
Fyodor Dostoevsky

Contents

1	Introduction	1
2	Motivation	3
3	Background and Related Work	4
3.1	Monte Carlo Estimation	4
3.2	Markov Chains	4
3.3	Cython	6
3.3.1	Cython functions	6
3.3.2	Compilation and Distribution	6
3.4	Recurrent Neural Networks(RNN)	7
3.5	Long Short Term Memory(LSTM)	7
3.6	Probabilistic Password Models	8
3.7	Previous Works	10
3.7.1	Classical Works	10
3.7.2	Modern	10
4	Methodology	11
4.1	Optimizing FLA sampling process	11
4.1.1	Pure Python	11
4.1.2	Heap-item	12
4.1.3	Custom-heap	12
5	Experimental Setup	14
5.1	Dataset	14
5.2	Model and Training	14
5.3	Evaluation Metrics	14
5.3.1	Password Rank Estimation	14
5.3.2	Guess Number and Guessing Percentage	15
5.3.3	Estimation with Monte Carlo	15
5.4	Implementation Details	16
5.4.1	Explanation	16
5.4.2	Caching step	22
6	Results and Analysis	24
6.1	Figures	24
6.2	Tables	25
7	Conclusion	27

8	Appendix	28
.1	UML	28
.2	Extended FOL	28
	Bibliography	30

Chapter 1

Introduction

In the field of probabilistic password-guessing models, evaluating a password's resistance to guessing attacks requires estimating its *guess number*. The guess number represents how many attempts an attacker would need, on average, before generating or matching a given password.

Computing this quantity exactly is infeasible for modern probabilistic models, as the password space is extremely large. For this reason, Monte Carlo estimation is employed to obtain an approximate rank without enumerating the full distribution. However, Monte Carlo estimation places strong computational requirements on the sampling procedure, which often becomes a bottleneck in practice.

The MAYA framework builds upon Monte Carlo methods to evaluate password-guessing models efficiently. Monte Carlo techniques, especially those derived from Dell'Amico and Filippone [2], offer effective rank estimation but require very large sample sizes to achieve stable accuracy. This, in turn, demands highly efficient sampling and memory management.

During the development of this thesis, the primary challenge was to optimize MAYA's sampling system used in the Fast, Lean and Accurate (FLA) password model. The original implementation relied heavily on Python data structures, which resulted in excessive memory usage due to object overhead. Because sampling dominates the cost of Monte Carlo estimation, reducing memory consumption and execution time became essential for enabling larger sample sizes and faster evaluations.

This thesis presents several contributions aimed at improving the efficiency of password rank estimation:

- An implementation of a Cython-based optimized heap item (`heap-item`), which reduces memory consumption while maintaining compatibility with Python's `heapq` module.
- A custom heap implementation (`heapcy`), designed to maximize performance by storing only file offsets rather than full strings, resulting in significant memory savings and faster operations.
- An integration of these optimizations into the MAYA framework to improve Monte Carlo rank estimation.
- A large-scale Monte Carlo experiment demonstrating that evaluation-estimation remains reliable up to $x = 5 \cdot 10^8$, far beyond the 10^7 tested in the original FLA paper.

The remainder of this thesis is structured as follows. Chapter 2 presents the motivation for this work, discussing the importance of password strength evaluation and the limitations of existing sampling approaches. Chapter 3 provides background material and an overview of related work in password-guessing models and Monte Carlo estimation. Chapter 4 describes the methodology and the proposed optimizations in detail. Chapter 5 explains the experimental setup used for evaluating the new implementations. Chapter 6 presents and analyzes the results. Finally, Chapter 7 concludes the thesis and outlines directions for future research.

Chapter 2

Motivation

Human-chosen passwords remain one of the most widely used authentication mechanisms, yet they continue to exhibit significant weaknesses. Password leaks occur frequently and at large scale, enabling attackers to build increasingly accurate probabilistic models that capture human password-selection patterns. Understanding how resistant user-chosen passwords are to such guessing attacks is therefore essential both for security research and for practical password-strength meters.

Modern password-guessing models estimate the probability of strings and generate guesses in decreasing likelihood order. A password’s *guess number* is defined as the number of guesses an attacker would need before encountering it in this ranked list. Determining this value exactly is infeasible for models with exponentially large output spaces. Monte Carlo estimation offers a tractable alternative, providing unbiased estimates of guess numbers without requiring full enumeration.

However, Monte Carlo estimation imposes significant computational demands. Accurate estimation requires very large sample sizes, often on the order of millions or tens of millions of generated passwords. Since each sample requires updating heap structures, memory footprint and sampling speed become the primary bottlenecks in frameworks such as MAYA. Monte Carlo also has a problem when estimating low probability passwords, as demonstrated in [6] there are ways to mitigate it, but they were not implemented in this thesis since they do not change the final result very much.

The Fast, Lean and Accurate (FLA) model is specifically designed to operate under strict memory and latency constraints. Its efficiency makes it suitable for client-side password-strength estimation. Nevertheless, its Python-based sampling system in MAYA suffers from high memory usage due to Python object overhead, which limits the size of the evaluation size and slows down the evaluation process.

The motivation behind this thesis is to extend the scalability of Monte Carlo estimation for password-guessing models by optimizing the underlying sampling mechanism. By designing more memory-efficient heap structures and reducing Python overhead through Cython, we enable substantially larger sample sizes and faster evaluation, both of which improve the accuracy and practicality of password-strength modeling.

Chapter 3

Background and Related Work

3.1 Monte Carlo Estimation

Given an analytically complex problem to solve, the use of Monte Carlo estimation provides a value close to the true solution, despite being analytically simpler. If a sample of n items is drawn from the main problem, we then perform a sum over the values, then divide at the end by *#samples*. This will give us an approximation of the expected value of our original problem. Let us say that we do not know the probability of a coin toss, we can use Monte Carlo to estimate this probability:

Given a Bernoulli random variable $X \in \{0, 1\}$

$$\hat{p} = \frac{1}{n} \sum_{i=0}^{n-1} X_i$$

As shown in the formula, above and according to the weak law of large numbers ¹

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n X_i = 0.5$$

3.2 Markov Chains

Markov chains provide a framework through which we can model sequential events. It works as follows: we have a set of states for each state we have associated a probability distribution of going to another, that can be easily represented with a matrix. To formalize this we can look at the below representation

Let $S = \{S_1, S_2, \dots, S_n\}$ be the state space.

Let P be the $n \times n$ transition matrix, where

$$P_{ij} = P(X_{t+1} = S_j \mid X_t = S_i)$$

We can see applications of this in fields such as: NLP², PPM³ etc... Markov Chains are often the basis for prediction models such as Probabilistic Password

¹The weak law of large numbers states that an observed sample average from a sample will be closer to the true population average as the sample grows larger. [3]

²Natural Language Processing

³Probabilistic Password Models

Models, but they are not enough to model these problems in fact in recent years we started using different kinds of Password-guessing models, like LLM based ones .

3.3 Cython

Cython (not to be confused with CPython) is a superset of Python. It is a compiled language, rather than interpreted, and allows for the creation of C/C++ extensions for Python with ease. The typical workflow is as follows: We first define compiler directives (starting with `#`). These tell the compiler how to translate the code to C/C++ and handle bindings. We then write the logic using Python-like syntax.

3.3.1 Cython functions

Cython provides three types of function declarators:

- **cdef**: Defines a C-only function. These are very fast but cannot be called from external Python code. By default, they act like standard Python functions and hold the [GIL](#)⁴, allowing access to Python objects. However, if defined with `nogil` (e.g., `cdef void func() nogil:`), they release the GIL; in this mode, Python structures cannot be accessed.
- **cpdef**: Creates two versions of the function: a C version for use within Cython and a Python wrapper for external use. This offers the best of both worlds but introduces slight overhead compared to pure `cdef`.
- **def**: These are standard Python functions. They are accessible from anywhere but incur the standard Python function call overhead.

This distinction allows us to write performance-critical loops/operations in `cdef` functions while keeping the interface accessible via `cpdef` or `def`.

Caller	Can call cdef?	Can call cpdef?	Can call def?
Cython (cdef/cpdef)	Yes	Yes	Yes
External Python	No	Yes	Yes

Table 3.1. interaction matrix between Python and Cython execution contexts. `cdef` functions optimize performance by bypassing the Python interpreter but are inaccessible to external Python modules

3.3.2 Compilation and Distribution

Cython's compiler will output `.whl` files that then can be imported into Python files like normal. The problem is that it can be burdensome to compile for every Python Version Header with only the Cython compiler. Therefore we opted to use [cibuildwheel](#) which when integrated with Github's CI/CD this facilitates software distribution.

⁴The Global Interpreter Lock is a Python feature which was introduced to eliminate race conditions in parallel operations on objects, Python needs it because objects are tracked by the Garbage collector by keeping count of the references of said object when the references reach 0 then the garbage collector frees memory, if there are parallel operations on the same object this could lead to an early free inducing a use after free, or race conditions

3.4 Recurrent Neural Networks(RNN)

Models that have 3 layers⁵:

0. Input Layer X
1. Hidden Layer H
2. Output Layer Y

This is what happens at each step t : the input vector x_t is used by the RNN to update its hidden state h_t , with this function^[5]:

$$h_t = \sigma_h(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad (3.1)$$

W_{xh} is the matrix that represents the weights between the input layer and the hidden one^[5], W_{hh} is the weight matrix for the recurrent connection and b_h is the bias vector.

At the end the model will update its y_t state in this manner ^[5]:

$$y_t = \sigma_y(W_{hy}h_t + b_y) \quad (3.2)$$

W_{hy} is the matrix that represents the weights between the hidden layer and the output layer^[5], and b_y is the bias vector. The recurring part is the computation of the hidden state at each step. However, this model type does not have good "Memory" because, during the hidden state calculation, the gradient starts to "vanish." This means that the model cannot learn long sequences of words. To solve this problem, there are many evolutions of this kind of model.

3.5 Long Short Term Memory(LSTM)

This is an evolution of RNN's which mitigates the vanishing gradient problem by controlling it. They introduce the concept of these 3 gates:

0. input
1. forget
2. output

Each of these tell the model how much of the previous, input, hidden and output states to consider from the previous state. Even though we have this improvement over the base model, this is still not perfect as the vanishing can still occur, nevertheless it is better than the normal one^[5]. LSTMs are particularly effective for password modeling because they capture long-range dependencies better than normal RNNs or simple Markov Chain Models.

⁵This is a simplification ^[5]

3.6 Probabilistic Password Models

P.P.Ms are a necessary part of a modern login security system they are needed to evaluate the strength of the user's password, since today's attackers also use these models to crack passwords more efficiently. When defending they evaluate the strength of a given password by telling you how many attempts it would take to guess it this is known as the **Rank** of a password, the higher the rank the more secure the password. Attackers prefer to use these models instead of brute forcing. Brute forcing is a procedure in which we try every possible password to find the correct one e.g. "What is Roberto Di Rosa's Gmail password?"

Let's assume he used only characters from the english alphabet and that the password has a maximum length of 12 characters, so we know that there are 26 characters in the english alphabet let's start by the first:

0. a
1. b
2. c ...
3. aa
4. ab
5. ac

We will try until we reach our result, the problem with this approach is that we might have to try: $26^{12} \sim 95.43$ quadrillions times, which for any pc is very computationally demanding. We could use optimized versions like [John The Ripper](#), which speed up the brute forcing job but are still very slow compared to these models. These probabilistic models instead rely on something else, after training we take all of the tuples of

0. password
1. password's probability

and put them inside of a set Γ ,⁶ we can now find out the rank of a given password α .

$$R(\alpha) = |\{\beta \mid \beta, p_\beta \in \Gamma \wedge p_\beta > p(\alpha)\}| \quad (3.3)$$

This will give us the rank of a password, or more simply, the amount of attempts it would take to guess it, the main problem from this approach is its data intensity the set of strings can be really big, so to be faster we need to use Monte Carlo's estimation^{3.1}, we start by defining a set Θ , it will contain a sample of strings taken from a i.i.d.⁷ distribution Ψ based on the internal one of the model. Furthermore the larger the size of Θ the better the estimation as proved in [2], this will help us especially with Low-probability passwords since they have higher variance. In this paper, we implemented the version found in [2] without any improvements as seen in

⁶If the model we are working with is implicit, we don't have access to any distribution, therefore at first it's impossible to use calculate the rank of a password in these models, without any modification/new model.

⁷Independently, Identically Distributed we need this because if the distribution is not i.i.d. therefore we will not be able to get accurate estimates

[6] this has not impacted on the performance of the guess% estimation. To this day we have not found a way to better this variance, the latest attempts have yielded some results in lowering the variance therefore making montecarlo more accurate especially for Low-probability passwords, but they are not good enough yet. We can now Estimate the **Rank**, we define $\hat{R}(\alpha)$ as the estimation of the Rank for a given password α :

$$\hat{R}_{\Psi}(\alpha) = \hat{\mathbb{E}}_{\beta \sim \Psi} \left[\frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)} \right] = \frac{1}{n} \sum_{\beta \in \Theta} \frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)} \quad [2] \quad (3.4)$$

This is a good estimation but if we need to estimate a lot of passwords, this will not be fast since each time we need to sample for montecarlo and calculate the rank, so the solution is to precalculate the rank and samples so that we do not need to do it multiple times, so let $\mathbf{A}$ be the descending sorted array of probabilities for getting generated by the model for each sampled password, and let $\mathbf{C}$ be the array of the estimated ranks for each password.

Let $\mathbf{A} = [p(\beta_0), p(\beta_1) \dots, p(\beta_n - 1)]$

$$\text{Let } \mathbf{C}[i] = \frac{1}{n} \sum_{j=0}^i -1 \frac{1}{A[j]} = C[i - 1] + \frac{1}{n \cdot A[i]}$$

So given a password α to find its rank given $\mathbf{A}$ and $\mathbf{C}$, we need to reverse binary search in $\mathbf{A}$ so that we find the index i of the probability which is lower than $p(\alpha)$, We than take the value stored at $\mathbf{C}[i]$ which will be the $\hat{R}(\alpha)$, so we found our solution really fast. We measure the efficacy of the model by seeing in how many attempts it takes to guess its test-dataset completely, the formula to do this is very simple:

$$G(x) = \frac{|\mathcal{S}(x) \cap \mathcal{T}|}{|\mathcal{T}|} \quad (3.5)$$

$x \in \mathbb{N}$ is the number of guesses the model makes, $\mathcal{T}$ is the set of all the test-dataset passwords, $\mathcal{S}(x)$ is the top-x sampling function that gives us the top-x most probable strings to be generated by the model. $G(x)$ is the ratio of the intersection between the sample set and the test-dataset,

They work by getting trained on a dataset(e.g. RockYou,Libero,TaoBao,etc...), from these datasets they will take the most frequent passwords and they will create distributions for these passwords in a way as to reflect the frequency of characters all of this using one of these RNNs, Markov chains, PCFGs, LSTMS, Random Forests[7], They can be separated into 2 types:

0. **Trawling** are aimed guessing correctly as many passwords as possible[8] and are constrained by the amount of resources and the number of guesses itself.
1. **Targeted** are aimed at guessing the password of a single user given some specifics (e.g.: Name,Surname,Sex,Nationality,etc...)

3.7 Previous Works

3.7.1 Classical Works

Fast Accurate And Lean(FLA)

FLA is an LSTM model that is used to guess passwords and study how effective an attacker might be at cracking passwords. As evidenced in [4] the model is very good at guessing passwords compared to PCFGs, Markov Models and more heuristic methods like John The ripper. Although in recent years there has been a development in to password guessing, as in the emergence of new kinds of model like: RFGuess [7], PASS LLM [8] etc... The authors of FLA have done an excellent job at making it very "lean" so that it can run easily on the client side, making it a really good model for password strength metering on the client-side which was a novel idea at the time. They also used Monte Carlo estimation[2] to efficiently look at how many guesses it would take to guess all of the test dataset [4], and proved reliability up to 10^7 .

3.7.2 Modern

Password Guessing With LLMs

As seen in [8] they brought forward the concept of using LLMs for password guessing and obtained really good results in the evaluation of guesses. These LLMs can get to higher guessing numbers much faster than previous implementations, instead of previous models.

Chapter 4

Methodology

4.1 Optimizing FLA sampling process

In this section, we start by analyzing the problems we encountered with MAYA and discussing the solutions we found to the said problems, their strengths, and weaknesses.

The problem we encountered was the need to sample the top-k elements from the distribution within FLA. To address these limitations, we explore two distinct strategies: The first, *Heap-item*, focuses on reducing item size via Cython structs. The second, *Custom-heap*, introduces a file-offset mechanism to eliminate string storage entirely

4.1.1 Pure Python

The first problem we tackled was the original implementation of FLA’s sampling algorithm[4], which occupied a significant memory usage due to the usage of pure Python structures such as lists, tuples, strings, and floats. The main problem came from the fact that each item was saved as a tuple of str, float, which was appended into a list, and that list was manipulated with the `heapq` module. Figure 1 illustrates the significant memory consumption caused by the original technique. Python heap structures store each heap item as a fully-fledged Python object (tuple of string and float). This leads to high memory overhead due to:

0. object headers,
1. reference counting,
2. heap fragmentation,
3. garbage collector metadata,
4. repeated storage of identical strings.

As a result, storing 10^6 heap entries may require hundreds of megabytes, when combined with all of the other objects we keep in memory this results in excessive memory usage, on the order of 200 GB when sampling $5 \cdot 10^8$ making it impossible to scale Θ to the sizes required for big model Evaluation.

4.1.2 Heap-item

The first proposed solution was to build a [heap-item](#) custom module written in Cython thus maximizing performance and compatibility with Python. A significant limitation of this approach is that, similar to the pure Python implementation, each heap item remains a distinct object managed by the garbage collector., each heap item is an object that must be tracked by python's [garbage collector](#), nevertheless this approach helped us reduce memory consumption by $\sim 59.79\%$ instead of the pure python approach as seen in Figure 1, however further optimization was necessary, so we opted for a custom heap called [heapcy](#).

Using Cython allows us to replace Python objects with C-structured objects giving us improved memory usage while also giving us ease of integration in Python, we could have used C++ and then have written the integrations with [nanobind](#), but that would have taken considerably more time and the fast software distribution would have been more complicated. Since we use Cython we have access to [cibuildwheel](#) which when integrated with github CI/CD allows us to publish binaries to PyPi rendering software distribution really easy. These Cython objects:

- are allocated in contiguous memory,
- avoid Python reference counting,
- do not trigger garbage collection,
- store fields compactly (the struct is made using chars and doubles which is much smaller than normal python strings, and floats)

This reduces per-item memory usage tremendously. However, each item is still an Python object, so total memory still scales linearly with the number of stored items.

4.1.3 Custom-heap

Rather than simply implementing a heap in Cython, we adopted a different approach by storing only the string's position in a file as a reference. This method significantly reduced memory usage, as shown in Figure 1, and required only a single heap object instead of multiple instances as in previous implementations [4.1.1](#) and [4.1.2](#). We first copy everything which was guessed by the `complete_guessing` from the zipped output file to a tmpfile created in the `/tmp` directory. The copy is made with the stream-based copying, after copying we keep the tmp-file open and then we start to take the strings position with the `tell()` method, we then read the line and save it all in the heapcy. After the heap we finish saving all of the heap elements, we start taking all of the offsets in decreasing order by probability, into a list, which will be passed to the helper method included into heapcy with generatorm pattern which will occupy $O(1)$ of memory and it will give us the strings one by one in $O(1)$ since we use the `seek()` method, which in binary files is $O(1)$. The I/O overhead added by the reading/writing of the files does not make the program take more time since the Cython optimizations counter said overhead, in our testing we registered for a sampling of 100_000 items: 2 hours and 40 minutes for the unoptimized version and 2 hours and 37 minutes for the optimized one. Therefore we are confident to say that this optimized version is good enough since, it has given us a memory consumption improvement of $\sim 87.47\%$ over [4.1.1](#) and a $\sim 68.85\%$ over [4.1.2](#) While this solution offers superior memory performance, it is more complex to use compared to the

previous solutions, so the choice should depend on the specific requirements of the development environment.

Since we do not actually need strings in memory during the sampling process, we can write a password once to a binary file, and the heap can store only its integer file offset together with the probability value.

This design:

- avoids storing millions of Python strings,
- eliminates repeated allocations,
- stores heap entries as compact C structs,
- ensures that the heap consists of a single contiguous memory region,

The tradeoff is increased implementation complexity and the need for careful file handling, but the performance benefit is decisive.

Chapter 5

Experimental Setup

5.1 Dataset

For this paper we chose the [RockYou](#) dataset the same as in MAYA [1](split 80;20), for time reasons we couldn't test with other datasets. RockYou is the standard dataset used in nearly all password-guessing literature. Using it ensures comparability with previous work and aligns with the datasets used by MAYA and FLA. Its size and structure also provide statistically meaningful evaluation for both trawling and targeted password-guessing attacks.

5.2 Model and Training

As discussed, there was no time to implement the functions required to test with other models. Model wise we used FLA found in MAYA[1], the model was pre-trained on the RockYou training set with a maximum password length of 12 characters, using the `rq1` test configuration. This setup isolates the impact of the memory optimizations, allowing for a direct assessment of the sampling efficiency without confounding variables from differing model architectures.

To evaluate these models we need to see how accurate their guessing [4] is, the following section will show us how to do this in multiple ways.

5.3 Evaluation Metrics

5.3.1 Password Rank Estimation

Limitations of Deterministic Ranking.

While using the deterministic rank would give us the most precise result, it requires an enormous amount of resources since we first need to calculate the entire set of passwords yieldable by the model, so we opt for the estimation. As described above, to get our guessing percentage estimate we first need to implement the Monte Carlo seen in [2]. To do this we need 1 set:

$\Theta := \{(pwd, prob) \mid (pwd, prob) \in \text{getTheta}(\text{model}, \text{sampleSize})\}$ our Monte Carlo sample.

5.3.2 Guess Number and Guessing Percentage

The standard way to get the guessing percentage of a model is the empirical way:

$$G(x) = \frac{|\mathcal{G}_x \cap \mathcal{T}|}{|\mathcal{T}|} \quad (5.1)$$

- $\mathcal{G}_x$ is the set containing the top-x sampling,
- $\mathcal{T}$ is the test-dataset containing all of the strings in the test-dataset.

This division gives us the real ratio of how many passwords of the test dataset were guessed, which can be seen as a percentage in the last column of 2. The former approach was very slow, we needed to find another way to reach a good enough solution.

5.3.3 Estimation with Monte Carlo

We employ a Monte Carlo estimate (5.2) to get a result similar to (5.1). $\hat{R}$, represents the estimate of the rank (number of times the model needs to guess¹) to generate the password.

Thanks to this we require much less effort compared to the deterministic way, since all we need to do is to compute the Monte Carlo curve once per Model Checkpoint, save it to .csv file and then for each guess number apply the formula above and we will have our results significantly faster than the previous approach.

The former approach was very slow since we needed to take the top_k where k is the guess number [1].

There are two ways to calculate the Monte Carlo estimation both specified in [2] and here included:

Eq. Monte Carlo Estimation for the rank of a password.

$$\hat{R}_\Psi(\alpha) = \hat{\mathbb{E}}_{\beta \sim \Psi} \left[\frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \right] = \frac{1}{n} \sum_{\beta \in \Theta} \frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \quad (5.2)$$

Array sorted in descending order

$$\text{Let } \mathbf{A} = [p(\beta_1), p(\beta_2), \dots, p(\beta_n)] \quad (5.3)$$

Eq. Optimized version

$$\text{Let } \mathbf{C}[i] = \frac{1}{n} \sum_{j=1}^i \frac{1}{A[j]} = C[i-1] + \frac{1}{n \cdot A[i]}$$

Estimation of the guess number for a given x.

$$\hat{G}(x) = \frac{1}{|T|} \sum 1_{[\hat{R}(\alpha) \leq x]} \quad (5.4)$$

Eq. Variant of (5.4)

$$\hat{G}(x) = \frac{|\{\alpha \in \mathcal{T} \mid \hat{R}(\alpha) \leq x\}|}{|\mathcal{T}|} \quad (5.5)$$

¹As defined in [2]

5.4 Implementation Details

5.4.1 Explanation

`get_theta`

The implementation for FLA, is very simple we first call the `get_theta` function inside of which we instantiate a guesser object we then call the `OneBatchToControlThemAll` which will sample in batches from the model distribution calling `IIDSampleBatched` and then these samples are written to a csv file which is our `montecarlo_sampling` or Θ .

Algorithm 1: Hierarchical Password Generation Process

```

1 function GetTheta (path,  $\mathcal{E}$ );
   Input : Output directory path, Config dictionary  $\mathcal{E}$ 
   // 1. Factory method creates the Guesser object

2 guesser  $\leftarrow$  GuesserBuild( $\mathcal{E}$ );
   // 2. The object executes the batch processing

3 N  $\leftarrow$  self.n_samples;
4 guesser.OneBatchToControlThemAll(N, true, path);



---


5 function OneBatchToControlThemAll (n, path);
   Input : Total samples n, Output path path
6 BatchSize  $\leftarrow$  2048;
7 generated  $\leftarrow$  0;
8 while generated < n do
9   need  $\leftarrow$  min(BatchSize, n - generated);
   // Delegate math to the core sampler
10  batch_data  $\leftarrow$  IIDSampleBatched(need);
11  Append batch_data to CSV at path;
12  generated  $\leftarrow$  generated + need;
13  if generated (mod 10) = 0 then
14    | Print progress;

```

Algorithm 2: Core Monte Carlo Sampler

```

1 function IIDSampleBatched ( $n$ );
   Input : Batch size  $n$ 
   Output : List of (password, probability) tuples
2  $S \leftarrow$  array of  $n$  empty strings;
3  $L \leftarrow \vec{0}_n$  ; // Log-probability accumulator
4  $Finished \leftarrow \vec{False}_n$ ;
5 for  $t \leftarrow 0$  to  $MaxLen$  do
6   if all  $Finished$  then
7     break;
8    $X \leftarrow \text{Encode}(S)$ ;
9    $P \leftarrow \text{Model}(X)$  ; // Raw probabilities
   // Apply structural constraints
10   $M \leftarrow \text{GetValidNextMask}(S)$ ;
11   $P \leftarrow P \odot M$ ;
12  Normalize rows of  $P$  to sum to 1.0;
13  if  $t = MaxLen - 1$  then
14    Force  $P$  to select END token;
15   $next\_tokens \sim \text{Categorical}(P)$ ;
16  for  $i \leftarrow 0$  to  $n - 1$  do
17    if not  $Finished[i]$  then
18       $k \leftarrow next\_tokens[i]$ ;
19       $L[i] \leftarrow L[i] + \ln(P[i, k])$ ;
20      if  $k = END$  then
21         $Finished[i] \leftarrow \text{true}$ ;
22      else
23         $S[i] \leftarrow S[i] + \text{Decode}(k)$ ;
24 return zip( $S$ , exp( $L$ ));

```

__build_mc_curve

here we call the function `__build_mc_curve` which will build the mc-curve from the file saved by [algorithm 1](#) it will return two arrays, A and C, A will be the array containing the probabilities in descending order and C will be the array containing the cumulative sum of the estimated ranks, such that for each probability in A at index $i \in [0..n] \wedge n \in \mathbb{N}$ the estimated rank of said probability will be at $C[i]$ If the arrays were not saved as tensors they will be saved so that future runs may be even faster.

Algorithm 3: Orchestration logic for building the Monte Carlo Curve

```

1 function BuildMCCurve ( $\mathcal{E}$ );
   Input : Evaluation Dictionary  $\mathcal{E}$ 
   Output: Tensors  $A$  (Probabilities) and  $C$  (Guess Numbers)
   // Define paths based on base directory

2  $P_{raw} \leftarrow$  "mc_samples.gz";  $P_{curve} \leftarrow$  "mc_rank_curve.gz";
3  $P_A \leftarrow$  "A.pt";  $P_C \leftarrow$  "C.pt";
4 if files  $P_A$  and  $P_C$  exist then
   | // Fast Path: Load pre-computed tensors
5   |  $A \leftarrow$  Load( $P_A$ );  $C \leftarrow$  Load( $P_C$ );
6 else
   | // Check if raw sampling is required
7   | if  $P_{raw}$  does not exist then
8   | | MonteCarloSampling( $P_{raw}, \mathcal{E}$ );
   | | // Compute statistics from raw sample file
9   | ( $A, C$ )  $\leftarrow$  ComputeCurveStats( $P_{raw}$ );
   | | // Cache results for future runs
10  | Save( $A \rightarrow P_A$ ); Save( $C \rightarrow P_C$ );
11  | if  $P_{curve}$  does not exist then
12  | | SaveCurve( $P_{curve}, A, C$ );
13 return ( $A, C$ );

```

Algorithm 4: Calculation of statistical tensors from raw samples

```

1 function ComputeCurveStats ( $path$ );
   Input : Path to raw Monte Carlo samples
   Output: Tensor  $A$  (sorted probs), Tensor  $C$  (cumulative cost)
   // 1. Parse probabilities from file
2  $Probs \leftarrow$  Read column 1 from CSV at  $path$ ;
3  $A \leftarrow$  Tensor( $Probs$ );
   // 2. Sort descending (most likely first)
4  $A \leftarrow$  SortDescending( $A$ );
   // 3. Compute expected guesses (CCS'15 Section 3.2)
5  $N \leftarrow$  Length( $A$ );
   // Compute inverse probabilities
6  $C \leftarrow 1 \oslash A$ ;
   // Update C with cumulative sum:  $C_i = \sum_{j=0}^i (1/A_j)$ 
7  $C \leftarrow$  CumSum( $C$ );
   // Divide by n so we finish applying montecarlo
8  $C \leftarrow C \oslash N$ ;
9 return ( $A, C$ );

```

Algorithm 5: Serializes the computed curve to disk

```

1 function SaveCurve ( $path, A, C$ );
   Input : Output path, Tensors  $A$  and  $C$ 
2 Open file at  $path$  for writing;
3 foreach index  $i$  in  $A$  do
   | // Write tuple (probability, expected_guesses)
4   | Write row ( $A[i], C[i]$ ) to file;

```

tester

Algorithm 6: Orchestrator for Monte Carlo Testing Session

```

1 function StartMonteCarloTest ( $Name_{ckpt}$ );
   Input : Name of the checkpoint  $Name_{ckpt}$ 
   // 1. Model State Management

2  $Path_{file} \leftarrow JoinPaths(Dir_{ckpt}, Name_{ckpt})$ ;
3  $Loaded \leftarrow LoadCheckpoint(Path_{file})$ ;
4 if not  $Loaded$  then
5   | Print "Checkpoint missing, starting training...";
6   |  $StartTrain(Name_{ckpt})$ ;
7   |  $LoadCheckpoint(Path_{file})$ ;
8 Print "Checkpoint loaded successfully.";

   // 2. Prepare Baseline Samples (Top-K)

9  $P_{topk} \leftarrow "topk\_guesses\_str\_float.gz"$ ;
10  $\mathcal{E} \leftarrow InitEval(N_{samples})$ ;
11 if File at  $P_{topk}$  does not exist then
12   | Ensure directory exists;
13   |  $Generator \leftarrow SampleModel(0, \mathcal{E})$ ;
14   | Open  $P_{topk}$  for writing;
15   | foreach ( $pwd, prob$ ) in  $Generator$  do
16   |   | Write " $pwd\ prob$ " to  $P_{topk}$ ;
17   | Verify total lines written equals  $N_{samples}$ ;
18   |  $PostSampling(\mathcal{E})$ ;

   // 3. Select Iterator Source and Execute

19 if  $Config.TestTopK$  is true then
20   | // Use the Top-K file generated above
21   |  $Iterator \leftarrow GetGeneratorFromTopK(P_{topk}, Device)$ ;
22 else
23   | // Use the full test dataset (default)
24   |  $Iterator \leftarrow GetTestDataset()$ ;

23  $t_{start} \leftarrow Time()$ ;
24  $MonteCarloTest(\mathcal{E}, Iterator)$ ;
25  $t_{delta} \leftarrow Time() - t_{start}$ ;
26 Print "Test completed in: "  $t_{delta}$ ;

```

Algorithm 7: Rank Estimation via Monte Carlo Curve

```

1 function MonteCarloTest ( $\mathcal{E}, \mathcal{T}$ );
   Input : Config  $\mathcal{E}$ , Test Iterator  $\mathcal{T}$  (pairs of  $pwd, prob$ )
   Output: CSV file with estimated ranks  $\hat{r}$ 
   // 1. Retrieve statistical curve tensors
2 ( $A, C$ )  $\leftarrow$  BuildMCCurve( $\mathcal{E}$ );
3 Open output file for writing;
4 foreach ( $pwd, target\_prob$ ) in  $\mathcal{T}$  do
   // Find where target_prob fits in distribution A
   // Since A is descending, we search for insertion point
5    $idx \leftarrow$  BinarySearch( $A, target\_prob$ ) - 1;
6   if  $idx < 0$  then
7      $\hat{r} \leftarrow 0$ ;
8   else
9      $\hat{r} \leftarrow C[idx]$ ;
10  Write ( $pwd, \hat{r}, target\_prob$ ) to file;

```

Algorithm 8: Monte Carlo Guessing Curve Analysis

```

1 function LoadGuessNumbers (path);
   Input : Path to Monte Carlo CSV results
   Output: Sorted list of guess numbers  $G$ 
   // 1. Parse rank estimates from file
2  $RawRows \leftarrow$  Read CSV at path;
3  $G \leftarrow$  Extract column rank_hat from  $RawRows$ ;
   // 2. Sort ascending for binary search (CDF)
4  $G \leftarrow$  SortAscending( $G$ );
5 return  $G$ ;

```

```

6 function ComputeCrackRates ( $G, range, step$ ) modified from [8]'s source
   code;
   Input : Sorted guesses  $G$ , Range  $[min, max]$ , Step size
   Output: Vectors  $X$  (thresholds) and  $Y$  (crack rates)
   // 1. Generate logarithmic thresholds (base 2)
7  $X \leftarrow \{min + step \cdot 2^i \mid i \geq 0 \wedge val \leq max\}$ ;
   // 2. Inject specific high-value benchmarks
8  $Benchmarks \leftarrow \{10^6, 2.5 \cdot 10^6, \dots, 5 \cdot 10^8\}$ ;
9  $X \leftarrow$  SortAscending(Unique( $X \cup Benchmarks$ ));
   // 3. Compute CDF via Binary Search
10  $N \leftarrow$  Length( $G$ );
11  $Y \leftarrow$  EmptyList();
12 foreach  $val \in X$  do
   // Count elements  $\leq val$  (bisect_left)
13    $count \leftarrow$  Index of first element  $\geq val$  in  $G$ ;
14    $rate \leftarrow count/N$ ;
15   Append  $rate$  to  $Y$ ;
   // 4. Serialize statistics to TSV
16 Write ( $X, Y$ ) to Table_file_sample_size_N.tsv;
17 return ( $X, Y$ );

```

```

18 function MainProcess ();
19  $Sizes \leftarrow [10, 100, 1000, \dots, 10^6]$ ;
20  $PlotBuffer \leftarrow$  EmptyList();
21 foreach  $S \in Sizes$  do
22    $path \leftarrow$  f-string("mc_results_{ $S$ }.csv");
   // Load and Analyze
23    $guesses \leftarrow$  LoadGuessNumbers( $path$ );
24    $(x\_vals, y\_rates) \leftarrow$  ComputeCrackRates( $guesses, [1, MAX], 0.5$ );
25   Print Progress: "Sample Size:  $S$ , Rate:  $y\_rates[-1]$ ";
26   Append ( $x\_vals, y\_rates$ ) to  $PlotBuffer$ ;
   // Generate individual plot
27   Render single curve to  $png$ ;
   // Generate combined comparison plot
28 Render all curves in  $PlotBuffer$  to guess_pcg_combined.png;

```

5.4.2 Caching step

To test the estimation for the evaluation we need to first take the test-dataset and for each password we need to get the probability of it being generated by the model and for good purposes, we should also sort all of it. To do that, we employ our `heapcy`, which makes this process much less memory intensive. We then write everything to a compressed file, if so that we do not need to generate it every time, the problem stems from the fact that for each time the model is trained we need to do this generation since the model could have changed its internal distribution, to optimize this we could work in batches to get the probability of multiple passwords at the same time, and to write into a file. same with the heap, we could maybe work with multiple heaps and then merge them together into one, when finished, all of course using batches and the gpu.

Algorithm 9: Cached Test Dataset Retrieval

```

1 function GetTestDataset ();
   Output : Stream of  $(pwd, prob)$  tuples
2  $Path \leftarrow$  "test_dataset.csv.gz";
3 if File at Path does not exist then
   | // Expensive generation step
4    $Stream \leftarrow$  GenerateSortedTestSet();
5   Open  $Path$  for writing (compressed);
6   while Stream is not empty do
7   |   Write batch from  $Stream$  to  $Path$ ;
   |
   // Yield from the cached file
8 return Stream reading from  $Path$ ;
```

Algorithm 10: External Sort for Probability Dataset

```

1 function GenerateSortedTestSet ();
   Input : List of test passwords  $P_{test}$ 
   Output: Generator yielding passwords sorted by probability
2  $\mathcal{H} \leftarrow$  New Min-Heap;
3  $Tmp \leftarrow$  Create temporary binary file;
   // Phase 1: Calculate and Index
4 foreach  $pwd$  in  $P_{test}$  do
5   if  $pwd$  is valid then
6      $offset \leftarrow Tmp.tell()$ ;
7      $prob \leftarrow \text{GetProbability}(pwd)$ ;
       // Store raw data on disk
8     Write  $(pwd, prob)$  to  $Tmp$ ;
       // Store index in RAM (prob, file_offset)
9      $\mathcal{H}.Push((prob, offset))$ ;
   // Phase 2: Sort and Retrieve
10  $SortedOffsets \leftarrow$  Extract  $N$  largest from  $\mathcal{H}$ ;
11 Clear  $\mathcal{H}$  from memory;
12 foreach  $offset$  in  $SortedOffsets$  do
13   Seek  $Tmp$  to  $offset$ ;
14   Read  $(pwd, prob)$  from  $Tmp$ ;
15   yield  $(pwd, prob)$ ;
   // Cleanup runs in finally block
16 Delete  $Tmp$ ;

```

Chapter 6

Results and Analysis

6.1 Figures

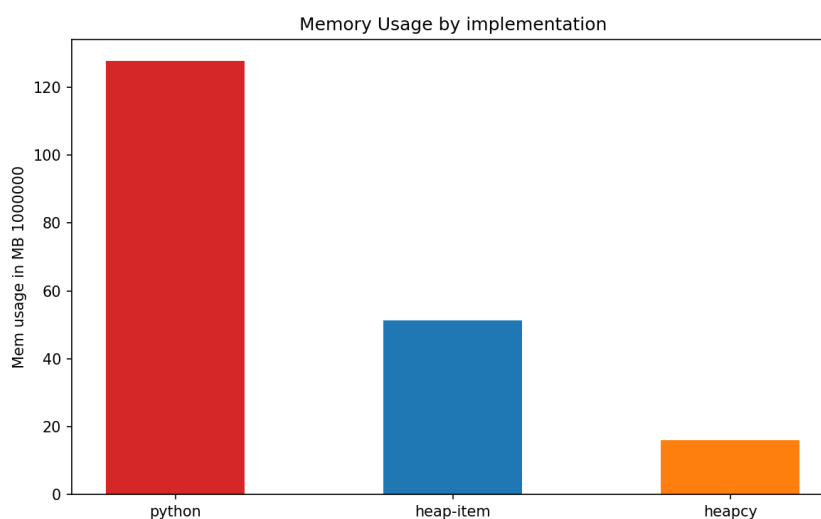


Figure 1. Comparative analysis of memory consumption across the three sampling strategies for a batch of 10^6 items. The **heapcy** (Custom-heap) implementation demonstrates a $\sim 87.47\%$ reduction in memory usage compared to the baseline **Pure Python** approach. This drastic drop is attributed to the file-offset mechanism (Section 4.1.3), which eliminates the overhead of storing string objects in RAM. The reduction in memory is not only an efficiency gain; it changes the problem class from 'requires a supercomputer' to 'runs on a laptop'.

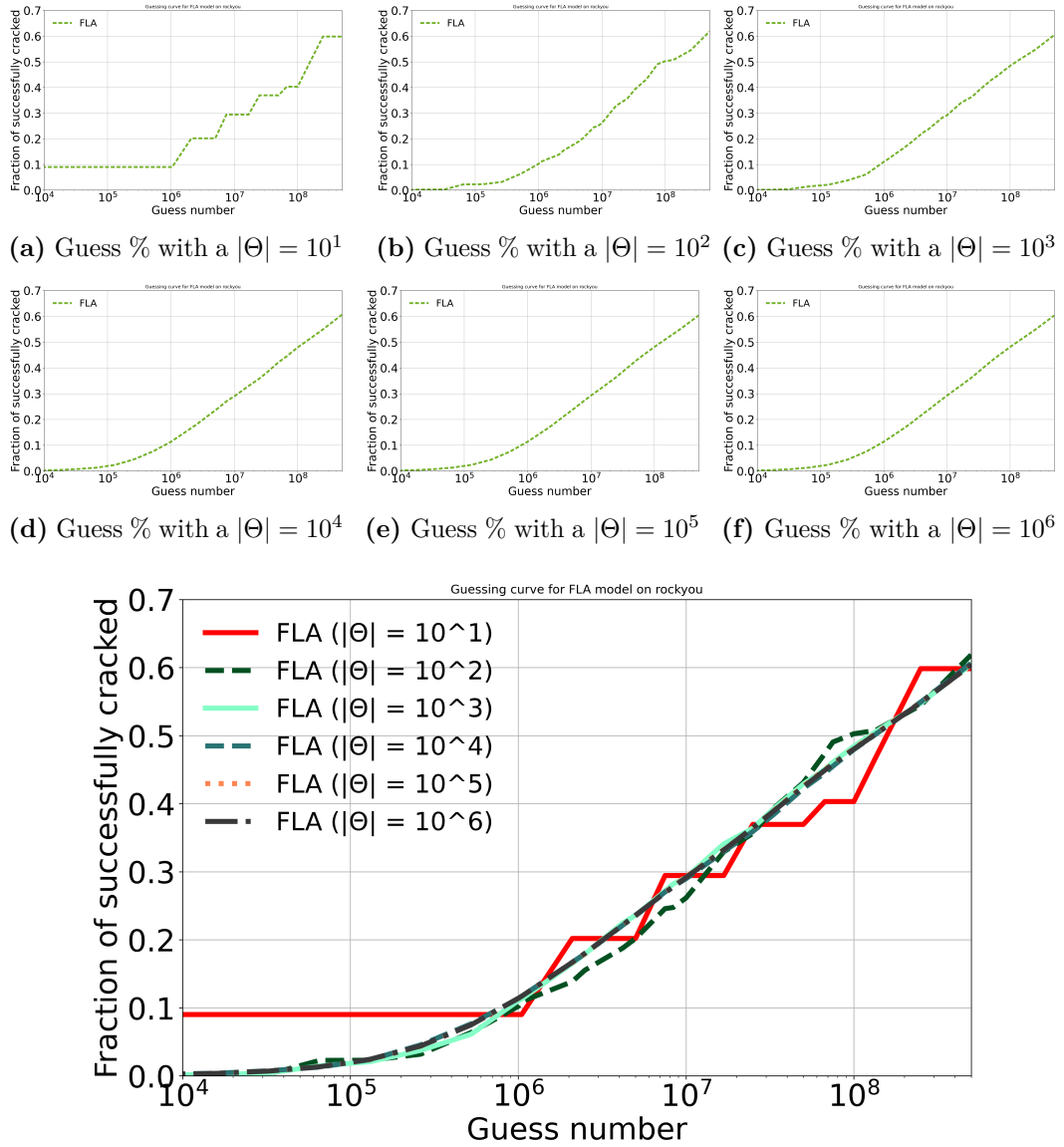


Figure 3. Convergence of the Monte Carlo rank estimation as the sample size $|\Theta|$ increases. Subfigures (a) through (d) illustrate that at lower sample sizes ($10^1 - 10^3$), the estimation exhibits high variance (noise). As $|\Theta|$ approaches 10^5 and 10^6 (f), the curve stabilizes, confirming that the memory optimizations allow for sample sizes large enough to produce deterministic-like accuracy.

6.2 Tables

#Item	Pure Python	Heap Item	Heapcy
10^6	127.72	51.36	16.00
10^5	12.77	5.13	1.60

Table 1. Size occupied in MB

As can be seen the gains have the same rate at any amount of items, so by using 10x the amount of items we will have 10x the amount of memory used.

Guess #	Rate 10^1	Rate 10^2	Rate 10^3	Rate 10^4	Rate 10^5	Rate 10^6	MAYA %
$1.0 \cdot 10^6$	9.01%	10.22%	11.03%	11.35%	11.37%	11.40%	11.40%
$2.5 \cdot 10^6$	20.19%	15.54%	17.91%	17.95%	17.97%	18.02%	18.04%
$5.0 \cdot 10^6$	20.19%	20.19%	23.60%	23.49%	23.57%	23.61%	23.66%
$7.5 \cdot 10^6$	29.43%	24.57%	27.17%	27.02%	26.94%	26.90%	26.94%
$1.0 \cdot 10^7$	29.43%	26.15%	29.15%	28.97%	29.25%	29.16%	29.20%
$2.5 \cdot 10^7$	36.95%	35.63%	36.41%	35.91%	36.42%	36.45%	36.48%
$5.0 \cdot 10^7$	36.95%	43.09%	42.88%	42.23%	42.59%	42.47%	42.49%
$7.5 \cdot 10^7$	40.31%	49.05%	46.09%	45.50%	45.84%	45.80%	45.82%
$1.0 \cdot 10^8$	40.31%	50.26%	48.43%	47.95%	48.11%	48.02%	48.04%
$2.5 \cdot 10^8$	59.84%	54.45%	54.74%	54.92%	54.85%	54.83%	54.85%
$5.0 \cdot 10^8$	59.84%	61.87%	60.45%	60.65%	60.35%	60.42%	60.47%

Table 2. Stability of the Guessing Percentage ($G(x)$) across varying Monte Carlo sample sizes. The data indicates that while estimates for high-probability passwords stabilize quickly (at $|\Theta| = 10^4$), accurate estimation for lower-probability tails (higher Guess Numbers) requires the massive sample sizes ($|\Theta| \geq 10^6$).

Chapter 7

Conclusion

In Conclusion we managed to optimize the sampling of FLA, added montecarlo to MAYA and we proved with results how the montecarlo estimation is very good at estimating the evaluation. We went beyond FLAs original paper by proving the result up to $5.0 \cdot 10^8$ this may imply that we do not need to the evaluation deterministic enumeration, by using enumeration, instead we can use montecarlo to see how good a model really is. Being able to extend Monte Carlo sampling to $|\Theta| = 5 \cdot 10^8$ is important, since the accuracy of it is entirely dependent on the size of $|\Theta|$. The larger the sample size the smaller the variance in the tail of the distribution (Low probability passwords), which is where many real passwords reside. Our optimizations therefore enable a better evaluation of password-guessing models without requiring enormous amounts of memory usage. In future works we should expand this method to every model implemented inside of MAYA, and try montecarlo for even larger numbers, since the new optimization allows us to enumerate up to even bigger numbers without incurring into big performance issues. We should as well optimize the caching part [10](#), since for now it is all sequential and takes a considerable amount of time $\sim 4hr$ for RockYou test-dataset $\sim 4.0 \cdot 10^6$ passwords. We could also improve the heapcy implementation to have a dynamic memory instead of the current implementation.

Chapter 8

Appendix

.1 UML

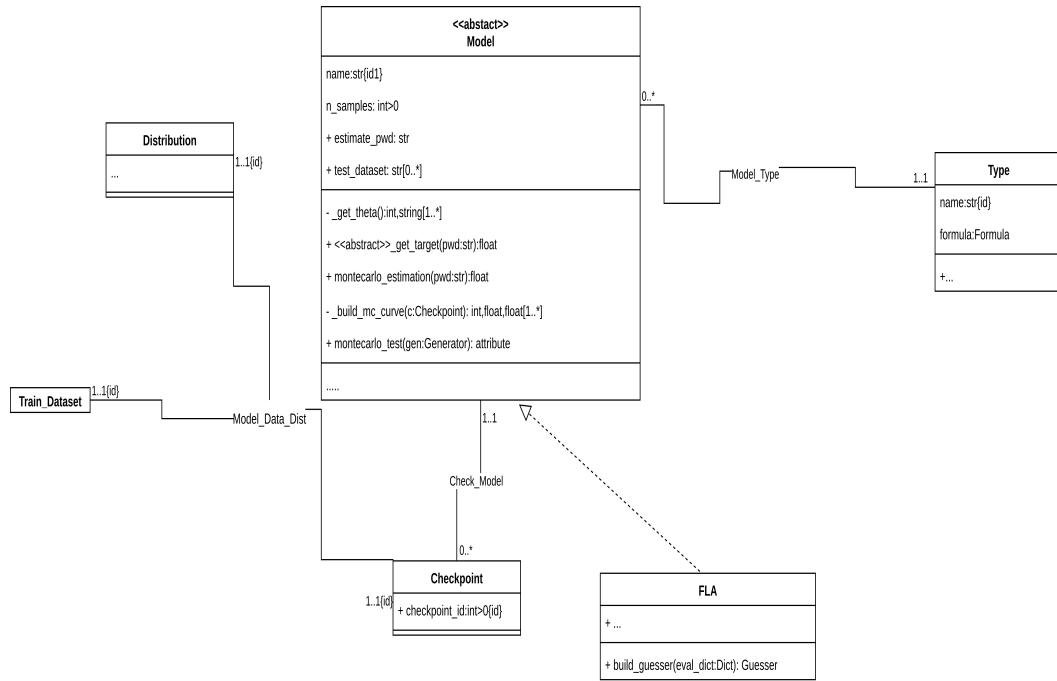


Figure .1. Analysis

.2 Extended FOL

Let $\mathbf{A} := \{(i, prob) \mid i \in \mathbb{N} \wedge (x, prob) \in sorted(\{x, p \mid \exists pwd \in \Theta \wedge x \in \mathbb{N} \wedge P(pwd, p) \wedge x = x + 1\}, sort_func) \wedge i = |\mathbf{A}| + 1\}$ (5.3)

$sort_func(t_1 : int \geq 0, real(0..1), t_2 : int \geq 0, real(0..1))) \rightarrow Bool :$
 $preconditions : None$
 $postconditions :$
 $|ret := \forall n_1, n_2,$
 $|Real(t_1, n_1) \wedge Real(t_2, n_2) \implies n_1 > n_2$
 $|return ret$

Let $\mathbf{C} := \{(i, n) \mid \exists x, prob \wedge i \in \mathbb{N} \wedge n \in \mathbb{R}_{>0} \wedge |A| > 0 \wedge n = \frac{1}{|A|} \cdot \sum_{x, prob \in A}^{i \leq x} \frac{1}{prob}\}$

Bibliography

- [1] CORRIAS, W., DE GASPARI, F., HITAJ, D., AND MANCINI, L. V. MAYA: A unified benchmarking framework for generative password-guessing models. In *2026 IEEE Symposium on Security and Privacy (SP)* (2026). ArXiv preprint arXiv:2504.16651v2. Available from: <https://arxiv.org/abs/2504.16651v2>.
- [2] DELL'AMICO, M. AND FILIPPONE, M. Monte carlo strength evaluation: Fast and reliable password checking. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security (CCS'15)*. ACM Press (2015). Available from: <https://dl.acm.org/doi/pdf/10.1145/2810103.2813631>.
- [3] LOÈVE, M. *Probability Theory I*. Springer, New York, 4th edn. (1977). Available from: https://archive.org/details/probabilitytheor0000loev_h6n0/mode/2up.
- [4] MELICHER, W., UR, B., SEGRETI, S. M., KOMANDURI, S., BAUER, L., CHRISTIN, N., AND CRANOR, L. F. Fast, accurate, and lean: A low-overhead, high-accuracy method for a-priori password strength estimation. In *25th USENIX Security Symposium (USENIX Security 16)* (2016). Available from: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/melicher>.
- [5] MIENYE, I. D., SWART, T. G., AND OBAIDO, G. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, **15** (2024). Available from: <https://www.mdpi.com/2078-2489/15/9/517>.
- [6] STANEK, M. Revisiting monte carlo strength evaluation. *arXiv preprint arXiv:2408.00124*, (2024). Available from: <https://arxiv.org/abs/2408.00124v1>, [arXiv:2408.00124](https://arxiv.org/abs/2408.00124).
- [7] WANG, D., ZOU, Y., ZHANG, Z., AND XIU, K. Password guessing using random forest. In *32nd USENIX Security Symposium (USENIX Security 23)* (2023). Available from: <https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-password-guessing>.
- [8] ZOU, Y., AN, M., AND WANG, D. Password guessing using large language models. In *USENIX Security Symposium (USENIX Security 25)* (2025). Available from: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-yunkai>.